Connect Four — Lab 4 VibeCoding

Chat history summary for submission | 11 October 2026

This document records the main guidance and decisions from the conversation about completing the Connect Four VibeCoding lab. It is a structured summary, not a verbatim export of every chat message.

1. Lab requirements

The student handout says to work individually, clone or fork the assigned repository, run the Python game, record a 10-second video before changes, use prompts to fix the broken code and add the README-listed features, record a second 10-second video after changes, and push the code to the student's own repository. Do not raise a pull request to the original SETAPESU26 repository.

Required deliverables in the Lab-4 folder

- Before video
- After video
- Updated code
- AI chat history exported as a document or PDF

2. Code changes and testing discussed

The project was worked on locally in ~/Desktop/15_connect_four. The discussed code changes included:

- Board win detection: include horizontal, vertical, and both diagonal directions.
- Input handling: non-numeric input is rejected; invalid column numbers and full columns are handled.
- AI move selection: take an immediate winning move when available; otherwise block an opponent's immediate win; otherwise choose a legal move.
- Game feedback: print a message after a successful disc placement.
- The student reported that diagonal/win checks and AI win/block tests passed.

3. Recording the AFTER video

The student completed the after video. For a normal gameplay demonstration, the suggested inputs were:

- 4 — demonstrate a valid move and disc-placement feedback.
- abc — demonstrate non-numeric input handling.
- 0 and 8 — demonstrate invalid column handling.
- q — quit the game.

The AI moves automatically after a valid player move, so a single normal game may not reliably demonstrate every edge case. Separate terminal tests can be used for AI win/block behavior and prepared board states.

4. GitHub submission repository

The student created/opened the repository LAB_4_15_connect_four and created a Lab-4 folder containing README.md. A ZIP named connect_four_project.zip was successfully selected/uploaded through GitHub's browser upload interface. The student was then guided to upload the two video recordings and the exported chat-history PDF into the same folder.

Important repository distinction

The lab submission repository is separate from the project repository 15_connect_four. The handout requires pushing the updated code to the student's personal project repository as well as placing the deliverables in the Lab-4 folder of the lab submission repository.

5. Git push troubleshooting

A local commit was created in the lab submission repository, but pushing through Terminal failed with HTTP 403 / permission denied. The screenshots showed a fine-grained token had repository access configured, but authentication still failed. The recommended safe approach was to revoke any token exposed in a screenshot, then use a fresh token with access to the exact target repository and Contents: Read and write. The student chose to continue with browser uploads to get the deliverables submitted.

Security note: Personal access tokens are secrets. Never include them in screenshots or share them in chat; revoke any exposed token.

6. Files to have in Lab-4

- connect_four_project.zip — updated project code.
- Before recording (.mov or another video format).
- After recording (.mov or another video format).
- This PDF — AI chat history summary.

Before finishing, open the GitHub Lab-4 folder and verify that all required files appear there. Then check the separate project repository to confirm the updated code has been pushed, as required by the handout.